

Paso 9 - Métodos | Ejercicios

Los métodos van **dentro de la clase** y **fuera del** `main`. Por ahora todos llevan `static`.

Y la regla para decidir el tipo: si el método **hace** algo, es `void`; si **responde** algo, devuelve un tipo y termina en `return`.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B9_1_TresMetodos.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B9.1 - Los tres clásicos

Qué practicas: escribir métodos con parámetros, distinguiendo los que devuelven un valor (`return`) de los que solo hacen algo (`void`).

Escribe estos tres métodos y pruébalos desde el `main`:

```
static boolean esPar(int n)
static double areaCirculo(double radio)
static void saludar(String nombre, int veces)
```

- `esPar` se prueba en un `for` del 1 al 10, imprimiendo `1 es impar`, `2 es par` ...
- `areaCirculo` con radio 2 tiene que dar 12.57 (`%.2f`).
- `saludar("Ana", 3)` imprime el saludo tres veces.

Fíjate en la diferencia: los dos primeros **responden** (uno `boolean`, otro `double`) y el tercero **hace** (`void`).

B9.2 - La nota en palabras

Qué practicas: que un método devuelva un resultado en vez de imprimirlo, para poder reutilizarlo en distintos lugares.

Escribe `static String clasificar(double nota)` que devuelva "Excelente", "Muy buena", "Suficiente" o "Insuficiente" según el ejercicio B5.6.

Pruébalo con un arreglo de notas y un `for`:

```
5.5 -> Muy buena
3.2 -> Insuficiente
6.8 -> Excelente
4.0 -> Suficiente
```

Ojo: este método **devuelve** el texto, no lo imprime. El `System.out.println` va en el `main`. Así el mismo método sirve para imprimir, para guardar o para comparar.

B9.3 - La tabla, ahora como método

Qué practicas: mover a un método un `for` que ya escribiste, y una primera sobrecarga con distinta cantidad de parámetros.

Convierte la tabla de multiplicar del paso 7 en `static void tabla(int n)` y llámala tres veces seguidas, para el 3, el 7 y el 9.

```
Tabla del 3:
3 x 1 = 3
3 x 2 = 6
...
3 x 10 = 30

Tabla del 7:
...
```

Después agrégale un segundo método `static void tabla(int n, int hasta)` que llegue solo hasta cierto múltiplo. (Esto es **sobrecarga**, y es el paso 12: aquí solo pruébalo.)

B9.4 - Partir un programa en métodos

Qué practicas: dividir un programa en métodos con una responsabilidad cada uno, y que unos métodos llamen a otros en vez de repetir código.

Toma tu solución del ejercicio **B8.1** (las cinco notas) y pártela en métodos, sin cambiar lo que hace:

```
static double[] leerNotas(Scanner sc, int cantidad)
static double promedio(double[] notas)
static double maxima(double[] notas)
static int contarRojas(double[] notas)
static void imprimirInforme(double[] notas)
```

Dos reglas: `imprimirInforme` **llama** a los otros métodos en vez de volver a calcular, y el `main` tiene que quedar en **cuatro o cinco líneas** (crear el Scanner, leer las notas, imprimir el informe, cerrar el Scanner).

Compara el antes y el después: ¿cuál se entiende más rápido?

Este ejercicio es el ensayo del paso 10, donde estos métodos se van a mudar a su propia clase.

B9.5 - Desafío: métodos que se llaman entre ellos

Qué practicas: que un método use a otro método que ya escribiste, en vez de repetir la misma lógica dos veces.

Escribe:

```
static boolean esPrimo(int n)
static int contarPrimos(int hasta)
```

`contarPrimos` tiene que **usar** `esPrimo` en su `for`, en vez de repetir el código. Prueba que `contarPrimos(30)` da 10.

Después responde: si mañana cambias cómo se calcula `esPrimo`, ¿cuántos métodos tienes que tocar?

Cuando termines, sigue con el [paso 10](#).